

Informatique Tronc Commun

Devoir Surveillé 1

1 Récursivité

1. On donne la fonction suivante :

```
def mystere(n):
    if n == 0:
        return 0
    else:
        return n%10 + mystere(n//10)
```

- (a) Déterminer le résultat de l'appel `mystere(563)`.
 - (b) Expliquer ce que calcule cette fonction `mystere`.
2. On rappelle que, pour p premier, la valuation p -adique d'un entier n est l'exposant auquel p apparaît dans la décomposition en facteurs premiers de n . Écrire une fonction `valuation` prenant en argument un entier premier p et un entier n , et renvoyant la valuation p -adique de n . **Cette fonction devra être récursive.**

2 Chiffrement de César

Le chiffrement de César est une méthode de cryptographie élémentaire : pour coder un message, on décale chaque lettre d'un même nombre de rangs dans l'alphabet, nombre qu'on appelle la **clé**. Le message codé ainsi obtenu n'est alors plus compréhensible par quelqu'un ne disposant pas de la clé, et peut être décodé par quelqu'un en disposant. Afin de pouvoir décaler toutes les lettres, on considère que a est le successeur de z. Par exemple, en codant le message "exemple" avec la clé 4, on obtient "ibiqtpi". Dans la suite, on supposera que les chaînes de caractères considérées ne contiennent que les lettres minuscules de a à z ou des espaces.

1. On suppose que la liste `alphabet = ['a','b','c','d', ... , 'y','z']` est définie comme variable globale, ce qui signifie qu'on peut l'utiliser dans n'importe quelle fonction sans la prendre en argument. Cette liste permet d'associer chaque lettre à son indice, qui est un entier de 0 à 25.
 - (a) Écrire une fonction `int_vers_char` prenant en argument un entier i supposé entre 0 et 25, et renvoyant la lettre associée.
Ainsi, `int_vers_char(3)` doit renvoyer 'd'.
 - (b) Écrire une fonction `char_vers_int` prenant en argument un caractère c , et renvoyant l'entier associé. La fonction ne renverra rien si c est une espace.
Ainsi, `char_vers_int('y')` doit renvoyer 24
 - (c) Donner une nouvelle version de la fonction précédente fonctionnant par recherche dichotomique (on peut en Python comparer des caractères selon l'ordre alphabétique de la même façon qu'on compare des nombres).
 - (d) Combien de caractères du tableau `alphabet` sont comparés à c au maximum lors d'un appel de la version dichotomique de `char_vers_int(c)` ?
2. Écrire une fonction `code_char(c,cle)` prenant en argument un caractère c et renvoyant la lettre obtenu en décalant c d'un nombre de rangs égal à **clé**, ou renvoyant une espace si c est une espace.
Ainsi, `code_char('y', 5)` doit renvoyer 'd', et `code_char(' ', 4)` doit renvoyer ' '.
3. Écrire une fonction `code` prenant en argument une chaîne de caractère m et un entier **cle**, et renvoyant le message obtenu par chiffrement de César en partant de m et en utilisant la clé **cle**.
Ainsi, `code("top secret", 10)` doit renvoyer "dyz combod"
4. Si on connaît la clé, il suffit d'utiliser la fonction précédente avec la valeur opposée pour décoder. Pour décoder sans la clé un message suffisamment long, on peut essayer une attaque statistique. Si on suppose que le message est initialement écrit en français, la lettre la plus fréquente a de fortes chances de correspondre à "e" après décodage, ce qui permet de déterminer la clé.
 - (a) Écrire une fonction `indice_max` prenant en argument une chaîne m et renvoyant l'entier associé à une lettre de nombre d'occurrences maximal dans m .
Ainsi, `indice_max("huuhxu")` doit renvoyer 20;
`indice_max("dyz combod")` peut renvoyer 3 ou 14.
 - (b) En déduire une fonction `decode` prenant en argument une chaîne de caractère correspondant à un message codé, et tentant de le décoder par l'attaque statistique présentée.

3 Palindromes

- On dit qu'une chaîne de caractères est un palindrome si elle se lit identiquement dans un sens ou dans l'autre, à l'exemple du mot *ressasser*.
 - Écrire une fonction `est_un_palindrome` prenant en argument une chaîne de caractères et déterminant (en renvoyant `True` ou `False`) si c'est un palindrome.
 - Déterminer le nombre de comparaisons entre caractères effectuées par cette fonction.
- La longueur palindromique d'une chaîne de caractère est la longueur maximale d'une sous-chaîne palindrome de cette chaîne. Par exemple, `"truculent"` a pour longueur palindromique 3, atteinte pour la sous-chaîne palindrome `"ucu"`.
Écrire une fonction prenant en argument une chaîne de caractères `c` et renvoyant sa longueur palindromique. Le nombre de comparaisons entre caractères effectuées par cette fonction devra être inférieur à $\text{len}(c)^3$, ce qu'on justifiera.

4 Vote par approbation

Le vote par approbation est un système de vote recommandé par de nombreuses associations mathématiques et utilisé par des outils comme Doodle. Chaque votant choisit un sous-ensemble des candidats, et le candidat choisi le plus remporte l'élection. On représente en Python le bulletin d'un votant par une liste **sans doublon**, et un vote par une liste de tels bulletins.

On donne par exemple le vote suivant, impliquant 3 votants et 4 candidats :

```
L = [ ['A', 'C', 'D'], ['A', 'B'], ['A', 'D', 'C'] ]
```

- Quel est le candidat remportant l'élection dans cet exemple ?
- Dans cette question, on veut s'assurer qu'aucun bulletin ne contient de doublon.
 - Écrire une fonction `a_un_doublon` prenant en argument une liste, et testant (en renvoyant `True` ou `False`) si elle contient un élément apparaissant au moins deux fois.
Ainsi, `a_un_doublon` doit renvoyer `True` sur `['A', 'B', 'A']` et `False` sur `['A', 'B', 'C']`.
 - En déduire une fonction `vote_valide` prenant en argument une liste de listes, et testant si aucune de ces listes ne contient de doublon. Ainsi, `vote_valide` doit renvoyer :
`True` sur `[ ['A', 'C', 'D'], ['A', 'B'], ['A', 'D', 'C'] ]`
`False` sur `[ ['A', 'C', 'D', 'A'], ['A', 'B'], ['A', 'D', 'C'] ]`
- On veut à présent écrire une fonction déterminant le vainqueur de l'élection.
 - Écrire une fonction `depouillement` prenant en entrée une liste de bulletins (donc une liste de listes), et renvoyant un dictionnaire associant à chaque candidat son nombre de votes.
 - En déduire une fonction `vainqueur` prenant en entrée une liste de bulletins, et renvoyant le vainqueur de l'élection. Cette fonction devra donc renvoyer le résultat déterminé à la question 1 sur la liste `Lex`.
- On s'intéresse maintenant à la question de déterminer le bulletin d'un votant en fonction de sa liste de préférences. Une stratégie simple et efficace est la suivante : Soient F (pour favori) et C (pour challenger) les deux candidats qu'un votant pense être en tête pour gagner l'élection, F étant le vainqueur le plus probable. Le votant va alors voter pour tout candidat préféré à F, et voter pour F s'il est préféré à C.
Écrire une fonction `strategie` prenant en argument la liste de préférences d'un votant (dans l'ordre du plus au moins préféré), un favori estimé F et un challenger estimé C, et renvoyant le bulletin décrit ci-dessus.
Ainsi, pour une liste `Lprefs = ['Alice', 'Charlie', 'Dave', 'Bob']` représentant les préférences Alice > Charlie > Dave > Bob, `strategie` doit renvoyer :
`['Alice']` sur `Lprefs, 'Alice', 'Bob'`
`['Alice', 'Charlie']` sur `Lprefs, 'Charlie', 'Dave'`
`['Alice', 'Charlie', 'Dave']` sur `Lprefs, 'Bob', 'Charlie'`
- Le favori estimé et le challenger estimé peuvent provenir de sondages, qu'on peut représenter eux-mêmes comme des votes par approbation. Écrire une fonction `limite` prenant en argument la liste des listes de préférences de chaque votant, initialisant les bulletins avec uniquement le candidat préféré de chaque votant, calculant le favori et son challenger sur cette élection, modifiant les bulletins des votants en fonction de la stratégie ci-dessus, procédant à une nouvelle élection, et ainsi de suite jusqu'à ce que les bulletins des votants restent inchangés. La fonction renverra le vainqueur de l'élection lorsque cette situation d'équilibre est atteinte.
- On peut montrer que la fonction précédente renvoie le vainqueur de Condorcet lorsqu'il existe. S'il n'y a pas de vainqueur de Condorcet en revanche, la fonction ne va pas terminer, les favoris et challengers calculés parcourant indéfiniment un cycle. Modifier la fonction pour que dans ce cas elle renvoie les favoris apparaissant dans ce cycle.